Jour 2 : MLP et Keras

Construire et entrainer des reseaux avec TensorFlow/Keras

Architecture Multi-Layer Perceptron (MLP)

Un MLP est un reseau de neurones avec une ou plusieurs couches cachees entre l'entree et la sortie.

Structure generale

- **Couche entree** : n features
- **Couches cachees** : 1-N couches ReLU
- **Couche sortie** : 1-K neurones

Exemple MNIST

- Entree : 784 pixels
- Couche 1 : Dense(128, ReLU)
- Couche 2 : Dense(64, ReLU)
- Sortie : Dense(10, Softmax)

> **Avantage :** Les MLPs peuvent apprendre n'importe quelle fonction continue (theoreme d'approximation universelle).

🔧 Keras Sequential API

Keras est l'API hautes niveaux de TensorFlow pour construire des modeles rapidement.

PYTHON

```python
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(10, activation='softmax')
])
```

Types de couches

- **Dense** : couche connectee
- **Dropout** : regularisation
- **BatchNormalization** : normalisation
- **Activation** : fonction d'activation

Parametres couches

- **units** : nombre neurones
- **activation** : fonction activation
- **input_shape** : forme entree
- **kernel_regularizer** : L1/L2

⚙️ Compilation du Modele

Avant l'entraînement, il faut compiler le modele avec optimizer, loss et metriques.

PYTHON

```python
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Choix optimizer

- `adam` : Standard, tres bon
- `sgd` : Simple, controlable
- `rmsprop` : Bon pour RNN

Loss fonctions

- `binary_crossentropy` : 2 classes
- `categorical_crossentropy` : 3+ classes
- `mse` : Regression

→ Entraînement du Modele

L'entraînement utilise `model.fit()` avec donnees d'entraînement.

PYTHON

```python
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
```

Parametres fit()

- **epochs** : passages complets sur dataset
- **batch_size** : exemples par gradient (32, 64, 128 courant)
- **validation_split** : portion de validation (0.1-0.2)
- **verbose** : affichage progress (0, 1, 2)

> **Retour :** `history` contient les metriques d'entraînement pour chaque epoch.

Learning Curves

Les courbes d'apprentissage montrent comment la loss et accuracy evoluent pendant l'entraînement.

Ideal

- Train loss diminue progressivement
- Val loss suit train loss
- Pas d'overfitting
- Convergence stable

Problemes

- **Overfitting** : val loss > train loss
- **Underfitting** : loss reste haute
- **Divergence** : loss augmente
- **Oscillation** : learning rate trop eleve

> ⚠ **Conseil :** Visualiser les courbes pour diagnostiquer les problemes d'entraînement.

Regularisation

La regularisation previent l'overfitting en penalisant les modeles complexes.

Dropout

Desactive aleatoirement neurones

- Taux : 0.2-0.5
- Applique couches cachees
- Pas en couche sortie

L1/L2

Penalite sur magnitude poids

- L1 : sparse (zero many)
- L2 : ridge (small many)
- Penalite : 0.001-0.01

Early Stopping

Arreter entraînement si val loss n'ameliore plus

- Monitor : 'val_loss'
- Patience : epochs sans amelioration

- Recupere meilleur modele

Callbacks

Les callbacks permettent de modifier le comportement pendant l'entraînement.

```python
callbacks = [
    EarlyStopping(monitor='val_loss', patience=3),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5)
]
model.fit(..., callbacks=callbacks)
```

Callbacks usuels

- **EarlyStopping** : arret premature
- **ReduceLROnPlateau** : baisse learning rate
- **ModelCheckpoint** : sauvegarde meilleur
- **TensorBoard** : visualisation

Parametres EarlyStopping

- **monitor** : metrique a surveiller
- **patience** : epochs sans amelioration
- **mode** : 'min' ou 'max'
- **restore_best_weights** : recuperer meilleur

Evaluation et Prediction

Apres l'entraînement, evaluer le modele sur test set et faire des predictions.

PYTHON

```python
# Evaluation
test_loss, test_acc = model.evaluate(X_test, y_test)

# Prediction
predictions = model.predict(X_new)
classes = np.argmax(predictions, axis=1)
```

Metriques

- **Loss** : erreur prediction
- **Accuracy** : % predictions correctes
- **Precision/Recall** : pour problemes desequilibres
- **F1-Score** : harmonic mean precision/recall

Important : Toujours evaluer sur test set non utilise pendant entraînement.

Resume du Module 2

Construction

- Sequential API
- Couches Dense + activation
- Input/output shapes

Entraînement

- Compilation (loss, optimizer)
- fit() avec epochs/batch
- Callbacks et early stopping

Regularisation

- Dropout
- L1/L2
- Validation split

Evaluation

- Metriques test set
- Learning curves
- Predict() sur new data